

GEOME Backend Data Model Review and Migration Plan

GEOME Backend Data Model Review and Migration Plan

Date reviewed: 2026-08-17

This document is a code-grounded architecture review of the current geome-db backend and a migration plan toward a simpler, more standards-driven data model. It is documentation only. It does not propose immediate production code changes.

Sources Inspected

Local code and schema:

- fimsCreateTables.sql
- src/main/resources/network-config-repository-sql.yml
- src/main/resources/record-repository-sql.yml
- src/main/resources/geome-sql.yml
- docs/geome.json
- src/main/java/biocode/fims/config/**
- src/main/java/biocode/fims/models/**
- src/main/java/biocode/fims/repositories/**
- src/main/java/biocode/fims/service/**
- src/main/java/biocode/fims/query/**
- src/main/java/biocode/fims/rest/**
- src/main/java/biocode/fims/validation/**
- src/main/java/biocode/fims/ncbi/**
- scripts/**

External standards references:

- MIxS documentation:
<https://genomicsstandardsconsortium.github.io/mixs/>
- MIxS GitHub releases:
<https://github.com/GenomicsStandardsConsortium/mixs/releases>
- MIxS v7.0.1 schema:
<https://raw.githubusercontent.com/GenomicsStandardsConsortium/mixs/v7.0.1/src/mixs/schema/mixs.yaml>
- MIxS release process:
<https://genomicsstandardsconsortium.github.io/mixs/releasing/>
- MIxS editing/release policies:
<https://genomicsstandardsconsortium.github.io/mixs/policy/>

As of this review, the official GitHub release list shows MIxS v7.0.1 as the latest full release, and the tagged schema declares version: 7.0.1.

Executive Summary

GEOME already uses a hybrid storage model: stable relational columns for project, expedition, entity identifier, local identifier, parent identifier, timestamps, and access-related state, with record metadata stored as JSONB. The main source of complexity is not "JSON versus relational"; it is that the network configuration defines the physical entity tables under `network_<id>`, while project configurations act as overlays that choose fields, labels, lists, and validation behavior for projects.

The current "team" concept is not represented as a separate backend table. In the code, team-like behavior is implemented by `project_configurations` rows, especially rows marked `network_approved = true`. Reporting also treats `projects.config_id` as `teamId`.

A single canonical backend model is feasible, but it should be introduced incrementally through inventory, standards mapping, a canonical service/API adapter, and profiles. A storage migration should come later, after the team can prove API compatibility, validation equivalence, BCID preservation, and query performance on representative data.

The proposed direction should be adjusted in four ways:

1. Do not conflate the expeditions table with the configured Event entity.
expeditions are project upload/collection groupings with BCID roots; Event

- is a scientific sampling event entity in the configurable data model.
2. GEOME already uses JSONB for scientific record metadata. The migration is primarily about decoupling schema generation from communities/profiles, not simply adopting JSONB.
 3. MIxS should be a versioned standards source for applicable scientific metadata terms and validation, not the whole GEOME backend model.
 4. Teams should stop defining physical backend schema. They can become published validation/profile objects.

A. Current Architecture

Logical Configuration

The logical configuration system has two layers.

`NetworkConfig` represents the network's available entity model. It defines:

- entity concept aliases and concept URIs;
- entity parent-child relationships;
- worksheets;
- unique keys and uniqueness scope;
- attributes/fields, including URI, column label, data type, definition, group, format, delimiter, and internal status;
- validation rules and controlled vocabularies;
- expedition metadata properties.

The implementation is centered on `Config`, `NetworkConfig`, `Entity`, `DefaultEntity`, and specialized entity classes such as `TissueEntity`, `PhotoEntity`, `FastaEntity`, and `FastqEntity`.

`ProjectConfig` is an overlay on top of `NetworkConfig`. It does not freely define a new backend schema. During validation, `ProjectConfig.mergeNetworkConfig` pulls in network lists and expedition metadata, preserves network-defined entity properties, and copies immutable field properties from the network. Project fields are tracked primarily by URI through `ProjectAttribute`, so project-level configuration can change presentation and some validation flags, but cannot redefine core network field identity.

`PersistedProjectConfig` stores only the project-specific portion of that overlay in the `project_configurations.config` JSONB column. It is converted back into a full `ProjectConfig` by combining the persisted overlay with the current network

config.

Projects and Configurations

Projects are stored in the projects table. A project has:

- config_id, pointing to project_configurations;
- network_id, pointing to networks;
- ownership and access state;
- public/discoverable flags;
- publication and data GUIDs;
- Local Contexts and permit identifiers;
- citation and license metadata.

Project creation accepts either:

- an existing projectConfiguration id; or
- an inline projectConfig, which creates a new ProjectConfiguration row for that project.

This means project configurations can function as both reusable "team" configs and one-off project configs.

Current Network Concept

The schema supports multiple networks through the networks table and per-network physical schemas named network_<id>. The GEOME application is configured around a single active network id through `GeomeProperties.networkId` and `SingleNetworkFeature`, which rewrites routes for single-network deployments.

Several scripts and reports assume network_1, so although the data model has a network abstraction, operationally this repository is strongly shaped around one deployed GEOME network.

Team / Network-Approved Configuration

There is no separate teams table in the inspected code. The team-like object is a project_configurations row:

- network_approved = true makes a configuration available through GET /projects/configs?networkApproved.

- Only network admins can directly create project configurations through the configuration resource.
- Only network admins can change the networkApproved flag.
- Reporting accepts teamId, but filters using p.config_id = :teamId.

So, current "team" semantics are a naming/API layer over reusable project configuration ids.

Physical PostgreSQL Storage

The physical model has three parts.

Public relational tables:

- users: user identity, auth-related fields, contact and SRA submitter fields.
- networks: network metadata and config JSONB.
- network_config_history: audit history for network config changes.
- project_configurations: reusable or project-specific config overlays in config JSONB, plus network_approved.
- project_config_history: audit history for project config changes.
- projects: project metadata, access flags, network id, and config id.
- user_projects: project membership join table.
- expeditions: project-scoped expedition/upload grouping, persistent root identifier, visibility, public flag, and metadata JSONB.
- entity_identifiers: one root identifier per expedition and concept alias.
- sra_submissions: SRA submission tracking.
- OAuth, invitation, and template tables.

Per-network schema:

- PostgresUtils.schema(networkId) returns network_<id>.
- PostgresUtils.entityTable(networkId, conceptAlias) returns network_<id>.<conceptAlias>.
- createNetworkSchema creates network_<id> and network_<id>.audit_table.

Per-entity physical tables:

Each network entity gets one physical PostgreSQL table, created from the network config. Each table has:

- id SERIAL;
- local_identifier TEXT NOT NULL;
- expedition_id INT NOT NULL REFERENCES expeditions(id);

- data JSONB NOT NULL;
- tsv TSVECTOR;
- created;
- modified;
- unique (local_identifier, expedition_id);
- GIN indexes on data and tsv;
- triggers for full-text indexing, audit history, timestamps, and project last-modified tracking.

Child entity tables additionally have:

- parent_identifier TEXT NOT NULL;
- foreign key (parent_identifier, expedition_id) to the parent entity table's (local_identifier, expedition_id);
- index (parent_identifier, expedition_id).

Therefore entities are represented as separate dynamically generated physical tables, while their scientific fields are stored in shared JSONB structure inside each table. This is a combination of dynamic tables plus JSONB, not a fully normalized table-per-field model.

Current Configured Entity Model

The checked-in docs/geome.json network config defines these entities:

Entity	Type	Parent	Worksheet	Unique key
Event	DefaultEntity	none	Events	eventID
Sample	DefaultEntity	Event	Samples	materialSampleID
Diagnostics	DefaultEntity	Sample	Diagnostics	diagnosticID
Tissue	Tissue	Sample	Tissues	tissueID
Sample_Photo	Photo	Sample	sample_photos	photoID
Event_Photo	Photo	Event	event_photos	photoID
fastaSequence	Fasta	Tissue	none	identifier
fastqMetadata	Fastq	Tissue	none	identifier

The current entity list is not a pure MxS model. It is a GEOME/Darwin Core style field data model with specialized classes for tissues, photos, FASTA, and FASTQ metadata.

Parent/Child Relationships

Logical parent/child relationships are configured on `Entity.parentEntity`. Physical child tables store `parent_identifier` and enforce that parent within the same expedition through a composite foreign key to the parent entity table.

Queries use `JoinBuilder` to traverse the configured parent-child chain. Joins compare child `parent_identifier` to parent `local_identifier` and keep `expedition_id` equal. The query builder rejects unrelated entity joins.

This is a strict tree-like relationship model. It supports direct parent relationships and traversals through configured ancestors/descendants, but it is not a general relationship graph with relationship types, many-to-many edges, or cross-expedition links.

BCIDs and Identifiers

BCIDs are built from:

- an expedition/entity root identifier stored in `entity_identifiers.identifier`;
- the record suffix, usually the entity unique key URI value;
- a resolver prefix from properties.

`BcidBuilder` concatenates the root identifier with either the entity unique key or, for child entities without their own unique key, the parent unique key.

`RecordService` resolves a record by parsing the ARK, looking up the root identifier in `entity_identifiers`, loading the project config, and querying by expedition plus unique-key suffix.

Preserving BCIDs requires preserving root identifiers, suffix semantics, expedition mapping, concept aliases, and unique-key values.

Configuration Changes and Physical Database Changes

Network configuration changes can alter the physical database:

- `NetworkService.saveConfig` generates URIs, adds default rules, validates the network config, then calls `NetworkConfigUpdater`.
- New entities trigger `createEntityTables`.
- Removed entities trigger `removeEntityTables`.
- Existing entities preserve immutable data: parent entity, unique key, and attribute URIs.

Project configuration changes do not create physical tables directly, but adding entities to a project config can create new entity root BCIDs for every project using that config.

This is the current point where logical configuration and physical storage are tightly coupled: network entities are not just logical concepts; they are table definitions.

Validation at Ingest/Update

Ingest runs through `DatasetProcessor`, `DatasetBuilder`, `DatasetValidator`, and `RecordValidator`.

The builder reads workbook/data sources using the project config, materializes record sets by entity, fetches existing records when needed for unique rules, fetches parent records for child validation, and applies converters.

Validation applies rules from the effective project config:

- required values;
- data type and date/time format checks;
- unique values, including project-wide uniqueness;
- controlled vocabulary checks;
- parent identifier validation;
- record-type-specific rules for photos, FASTA, FASTQ, and tissues.

After validation, `PostgresRecordRepository.saveDataset` upserts records into the dynamic entity table. Data values are serialized into JSONB. Local identifiers and parent identifiers are duplicated into relational columns for identity, uniqueness, joins, and deletes.

REST API Dependence on Configuration

The API depends on configuration in several ways:

- Query parsing starts from the network config. If a query is scoped to exactly one project, `Query.setProjectConfig` swaps in the project config so output uses project-specific fields.
- `QueryBuilder` maps human column names to field URIs and emits SQL over `network_<id>.<conceptAlias>`.
- API results map stored URI-keyed JSONB records back to configured project column names.
- Export code chooses entities from config and has comments noting limitations when multiple unrelated parent entities exist.
- Record lookup by BCID depends on `entity_identifiers`, concept alias, project config, and unique key.
- Reporting uses network config, probes `network_<id>` tables, and uses `teamId` as `projects.config_id`.
- SRA submission code assumes `FastqEntity` and walks configured parent relationships from FASTQ metadata to parent material entities.

The API surface is therefore not just using config for labels; config determines available entities, legal joins, validation behavior, input templates, output headers, and physical SQL targets.

B. Current Architecture Diagram

The simple diagram in the prompt is close but incomplete. A more accurate view:

```

networks
  config JSONB -> NetworkConfig
    - available entities
    - base fields and URIs
    - base rules and lists
    - parent-child relationships
    - worksheets and record types
      |
      | NetworkService.saveConfig()
      | creates/drops entity tables
      v
network_<id> schema
  audit_table
  <conceptAlias> table per network entity
  id
  expedition_id -> expeditions.id

```

```

    local_identifier
    parent_identifier, for child entities
    data JSONB
    tsv
    timestamps

project_configurations
    config JSONB -> PersistedProjectConfig overlay
    network_approved=true -> reusable/team-like configuration
        |
        | projects.config_id
        v
projects
    project metadata, access, publication/local-context/permit fields
        |
        v
expeditions
    project-scoped expedition/upload grouping
    persistent root identifier
        |
        v
entity_identifiers
    expedition_id + concept_alias -> root identifier
        |
        v
records in network_<id>.<conceptAlias>
    BCID = resolver prefix + root identifier + local identifier suffix

```

The important correction is that configurations/teams do not create separate physical project or team schemas. Network config creates physical entity tables. Project configurations are overlays and reusable templates over those network entities.

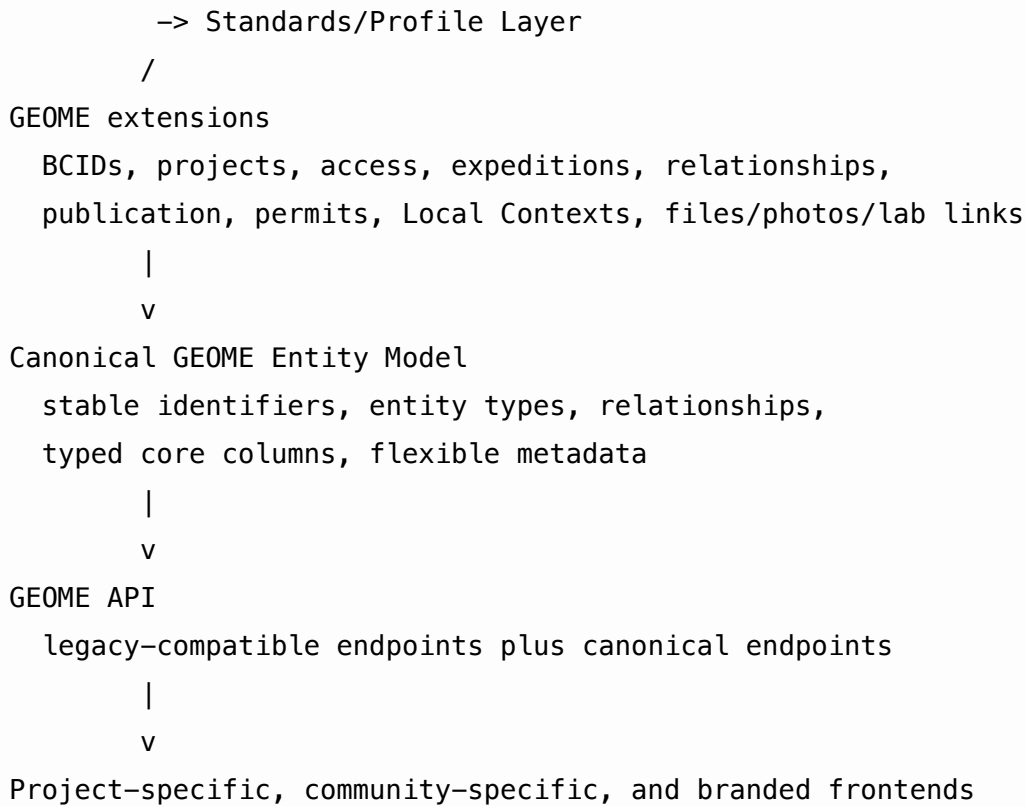
C. Proposed Architecture

Target Shape

```

MIXS versioned source
    terms, checklists, extensions, combinations, value rules
    \

```



Canonical Entity Concepts

The current checked-in config suggests the following canonical concepts, but the names should be confirmed through data inventory rather than imposed upfront:

- Project: keep as a relational administrative and publication unit.
- Expedition: keep as a project-scoped collection/upload grouping, but avoid treating it as the same concept as a sampling Event.
- Event: sampling/collecting event.
- Material entity: a sampled physical thing, covering current Sample and many Tissue/subsample cases.
- Material derivation: tissue, subsample, extraction, or other derivative of a material entity. This may be best modeled as an entity type plus relationship, not necessarily a separate hard-coded backend table per subtype.
- Observation/measurement/diagnostic: current Diagnostics.
- Media/resource: photos, documents, uploaded reports, external URLs.
- Sequence/derived data reference: FASTA, FASTQ metadata, ASV tables, analysis products, external lab results.
- Persistent sampling target/station: only promote to canonical entity if inventory shows repeated sampling locations/targets need identity independent of events.
- Permit/Local Contexts/publication artifacts: likely project-level or resource-level GEOME extensions, not MIxS-derived entities.

Hard-Coded Entities vs Entity Type + Metadata

Hard-coding separate backend entities for Sample, Tissue, Extraction, and Environmental Sample has advantages:

- simpler SQL for common workflows;
- easier targeted indexes and constraints;
- clearer API contracts for clients that know these concepts;
- easier migration from the current concept aliases.

It also has costs:

- more backend schema churn when communities add related material subtypes;
- pressure to keep adding special classes like `TissueEntity`;
- duplicated validation and import/export behavior;
- harder support for new communities without backend deployments.

A common canonical entity with `entity_type`, typed core columns, relationships, and profile-driven metadata has advantages:

- one storage and API pattern for material derivatives;
- profiles can distinguish sample, tissue, extraction, environmental sample, voucher, or subsample without creating tables;
- easier standards mapping and versioned validation;
- lower long-term maintenance.

Its risks are:

- weaker database constraints unless relationship rules are explicit;
- generic APIs can become vague;
- performance needs careful partial indexes and generated columns;
- migration must preserve legacy concept aliases and BCIDs.

Recommendation: introduce the canonical model as a service/API abstraction first. Do not immediately collapse storage. For future storage, use explicit canonical entity types plus typed relationship edges. Promote only stable, commonly queried attributes to columns.

Relational Core + Flexible Metadata

A practical future storage model would look more like this:

```
canonical_entity
  id
  project_id
  expedition_id
  legacy_network_id
  legacy_concept_alias
  entity_type
  local_identifier
  persistent_identifier_root
  bcid
  visibility/publication state
  created_at
  updated_at
  metadata JSONB
```

```
entity_relationship
  id
  project_id
  subject_entity_id
  object_entity_id
  relationship_type
  legacy_parent_identifier
  created_at
```

```
entity_resource
  id
  project_id
  entity_id
  resource_type
  external_id
  url
  file_id/blob_key
  metadata JSONB
```

This model keeps identifiers, relationships, project/expedition scope, access, and resource linkage relational. Scientific metadata remains flexible and profile-driven in JSONB. High-use terms can be indexed with expression indexes, generated columns, or promoted typed columns after usage data proves the need.

Compared with the current architecture:

- Current storage already uses JSONB for fields, so the biggest change would be replacing dynamic per-entity tables with one or a few canonical tables.
- Current parent-child links are strict table-level parent FKs. A canonical edge table would handle more relationship types and many-to-many cases but needs stronger application validation.
- Current queries are concept-alias-first. A canonical model should preserve legacy concept aliases through views or adapters while adding stable entity types.

Compared with fully normalized entity tables:

- A fully normalized model gives stronger typing and SQL constraints, but GEOME's scientific metadata varies by community, standard version, and profile. Fully normalized storage would likely recreate the current schema-management burden.
- Relational core plus JSONB better matches a metadata/identifier backend, as long as term definitions and validation are externalized and versioned.

JSONB is appropriate here, but only for flexible scientific metadata. It should not hide project membership, access control, BCIDs, entity identity, relationships, publication state, or resource linkage.

MIxS Role

MIxS can reasonably provide:

- scientific metadata terms and definitions;
- checklists for genomic sequence types;
- environmental extensions/packages;
- combinations of checklists and extensions;
- value patterns, enumerations, and some validation guidance;
- globally resolvable identifiers for terms, checklists, extensions, and combinations;
- a LinkML source schema and generated artifacts.

MIxS should not control:

- GEOME project organization;
- user access control;
- BCID/root identifier semantics;
- expedition/upload grouping;
- parent-child material lineage;

- GEOME publication workflow;
- Local Contexts and permit handling;
- photo/file storage;
- external laboratory resource linkage;
- all UI grouping and workflow decisions.

MIxS versioning should be explicit:

- pin a MIxS release per GEOME standards import, for example 7.0.1;
- store source, version, term id, term name, and schema path in a local term registry;
- store profile versions separately from MIxS versions;
- lock projects to a profile version at upload time;
- provide migration reports when a profile moves to a newer MIxS version;
- never silently change validation for historical project data because a new MIxS release exists.

MIxS is best treated as an external standards dependency feeding a GEOME standards/profile layer, not as the replacement for GEOME's domain model.

Profiles Instead of Schema-Defining Teams

The profile concept maps well to the existing `project_configurations` architecture because project configs already:

- select a subset of network entities;
- choose fields;
- add project/community validation rules;
- carry controlled vocabulary lists;
- drive worksheets/templates;
- can be made reusable with `network_approved`.

However, the current table and API mix several concepts:

- one-off project config;
- reusable team config;
- network-approved community template;
- effective project config after network merge.

Recommendation:

- Introduce profile terminology in the service/API layer before changing schema.
- Treat existing `network_approved=true` rows as published legacy profiles.
- Add inventory and version metadata before any migration.
- Long term, separate profiles, `profile_versions`, and `project_profile_assignments` from the effective config snapshot used by historical uploads.
- Profiles should constrain and extend a canonical model, not create new physical backend tables.

GEOME-Specific Extensions

These should remain GEOME-specific even if some metadata fields map to MIxS or Darwin Core:

- BCIDs and root identifier allocation;
- project ownership, membership, access, visibility, and publication state;
- expedition/upload grouping;
- parent-child material relationships and derivation lineage;
- Local Contexts linkage;
- permits;
- project data GUIDs and publication artifacts;
- photo/media/file storage and processing status;
- external lab identifiers and URLs;
- SRA submission state;
- FASTQ/FASTA/ASV/derived output linkage;
- project-specific validation requirements;
- UI worksheet/template organization.

Where a GEOME-specific field overlaps MIxS or Darwin Core, store a crosswalk rather than replacing the GEOME concept immediately.

Laboratory and Derived Data Relationships

GEOME should not become a LIMS. It should store identifiers, metadata, and links to external lab or derived-data resources.

Recommended model:

- Keep material lineage in GEOME: Sample -> Tissue/subsample -> Extraction or equivalent typed material derivatives.
- Store external lab references as resources attached to the relevant material entity or derivation: lab name, lab identifier, external URL, report file, submitted date, status, and minimal metadata.
- Store derived outputs as resources or derived-data entities: FASTQ references, FASTA sequences, ASV table references, analysis product URLs, taxonomic result files, and external accession identifiers.
- Do not store detailed pipeline execution steps, parameters, compute logs, or workflow provenance unless GEOME later explicitly takes on workflow-management scope.

This keeps GEOME's role focused on metadata, identifiers, validation, linkage, and publication.

D. Gap Analysis

Subsystem	Recommendation	Notes
projects table and project service	KEEP / MODIFY	Core administrative unit. Add profile-version linkage later.
expeditions table	KEEP / MODIFY	Preserve as upload/collection grouping. Clarify distinction from sampling Event.
entity_identifiers and BCID builder	KEEP / MODIFY	Preserve identifiers. Formalize suffix/root mapping before storage changes.
Network config as entity catalog	MODIFY	Useful as legacy catalog, but should stop being the long-term schema generator.
Network config creates/drops entity tables	DEPRECATE	Main coupling to remove after adapters and compatibility tests exist.

Subsystem	Recommendation	Notes
Project configurations	MODIFY	Good base for profiles, but needs explicit versioning and clearer semantics.
network_approved team behavior	DEPRECATE	Rename/recast as published profile/community template.
Per-entity dynamic tables	MODIFY / REPLACE LATER	Keep during early phases. Replace only after canonical API and migration tests.
Record data JSONB	KEEP / MODIFY	Already appropriate for flexible metadata. Add standards registry and targeted indexes.
Parent parent_identifier model	MODIFY	Preserve for legacy. Add typed relationship model for richer lineage/resource links.
Query DSL and QueryBuilder	KEEP / WRAP	Needed for compatibility. Add canonical query service on top.
Reporting controller	MODIFY	It embeds team/config and entity assumptions. Use inventory/canonical view later.
Validation rules	KEEP / MODIFY	Reuse rule framework, but feed it from profile + standards registry.
Specialized Tissue/Photo/FASTA/FASTQ classes	MODIFY	Keep behavior, but migrate concepts toward material/resource/derive data model.
SRA submission support	KEEP / MODIFY	Useful integration. Keep out of generic canonical entity core where possible.

Subsystem	Recommendation	Notes
Scripts assuming network_1	MODIFY	Operational artifact. Parameterize or document as single-network deployment detail.
Historical migrator scripts with old schemas	DEPRECATE	Keep for history only unless still used.
project_templates vs worksheet_templates naming drift	UNKNOWN / NEEDS INVESTIGATION	Schema and JPA names differ; verify migrations before cleanup.

E. Migration Roadmap

Phase 0: Architecture Discovery and Inventory

Build automated reports before changing storage:

current GEOME field
current entity
current configuration/team
usage count
MIXS equivalent
Darwin Core equivalent
proposed canonical term
migration status

Also inventory:

- current network configs and project configs;
- physical network_<id> schemas and table row counts;
- field URI usage in JSONB;
- concept aliases and casing;
- parent-child relationship depth;
- BCID roots and suffix patterns;
- API routes and reports that construct raw network/entity SQL.

Phase 1: Standards Layer, No Storage Change

Introduce a versioned standards registry:

- import MIXS terms/checklists/extensions from a pinned release;
- capture Darwin Core and GEOME-specific terms in the same registry;
- map current GEOME fields to standards terms where possible;
- add validation comparison tests but do not change production validation behavior yet;
- expose standards metadata to templates and future profile tooling.

Phase 2: Canonical Model Abstraction Over Current Storage

Create a service that reads current dynamic tables and returns canonical entities/relationships/resources.

This should:

- preserve current APIs;
- add new internal canonical DTOs;
- map legacy concept aliases to canonical entity types;
- make BCID resolution go through one canonical identifier service;
- provide read-only endpoints or feature-flagged internal APIs first.

Phase 3: Profiles

Separate validation/profile behavior from physical schema generation:

- treat network_approved configurations as legacy published profiles;
- add explicit profile/version metadata;
- snapshot effective config at project upload time;
- allow profiles to reference MIXS package/version plus GEOME extensions;
- keep old project configs readable through a legacy adapter.

Phase 4: Storage Migration, If Justified

Only after Phases 0 to 3 show the path is safe:

- create relational core plus JSONB metadata tables;
- build legacy-compatible SQL views or API adapters;
- migrate a representative subset of projects;
- compare validation, export, reporting, and BCID resolution outputs;
- benchmark common queries and reports;
- consider dual-write for a bounded period if operationally necessary.

Phase 5: Legacy Cleanup

After compatibility is demonstrated:

- remove dynamic schema creation from normal workflows;
- freeze legacy network config edits;
- deprecate team/config naming in API responses;
- simplify reports and exports to use canonical services;
- retire old migrator scripts and hard-coded `network_1` assumptions where practical.

F. Risks

Data Migration Risk

Highest risk areas:

- URI-keyed JSONB fields with historical changes;
- concept alias casing and table names;
- existing project configs that depend on network config merge behavior;
- parent-child links encoded by local identifiers rather than stable entity ids;
- specialized records such as photos, FASTA, FASTQ, and tissue-generated IDs.

API Compatibility Risk

Existing clients may depend on:

- concept aliases;
- worksheet names and CSV/TSV headers;
- query DSL column names;
- BCID strings;
- `teamId` meaning `config_id`;
- export shapes and SRA file generation.

Compatibility should be proven with regression tests and adapters before storage changes.

Performance Risk

Current per-entity tables have narrow row sets and GIN indexes per entity. A single canonical entity table could grow much larger and require:

- partial indexes by entity type and project;
- expression indexes for high-use JSONB terms;
- generated columns for common filters;
- careful query planning for reports;
- benchmarked migration decisions.

Identifier Risk

BCIDs are central. The migration must preserve:

- root identifiers in `entity_identifiers`;
- local identifier suffixes;
- root-plus-suffix construction;
- old ARK resolution;
- project/expedition/entity association.

Do not regenerate identifiers as part of storage migration.

Validation Risk

MIxS mapping can change requiredness, controlled values, cardinality, and syntax. GEOME should avoid changing historical validation semantics silently. New profile versions should be explicit and comparable.

Configuration Compatibility Risk

Old project configs may rely on implicit network fields, network rules converted to project rules, and incomplete overlays. They need a legacy effective-config adapter and test fixtures.

G. Initial Engineering Tasks

1. Add an inventory report for field/entity/config usage. Output current field URI, column, entity, config id, network-approved status, project count, record count, MIxS/Darwin Core candidate, proposed canonical term, and migration status.
2. Add a schema and SQL dependency report. List all places that construct `network_<id>.<conceptAlias> SQL`, depend on `network_1`, or treat `config_id` as `teamId`.

3. Create a versioned standards registry prototype. Start as read-only tables or checked-in seed JSON. Include MIXS source, version, term id, name, definition, range, enum/pattern, and package membership.
4. Map the top 50 current GEOME fields. Use inventory counts to map high-impact fields first to MIXS, Darwin Core, or GEOME-specific terms.
5. Add profile/version metadata without changing validation behavior. Treat existing `network_approved=true` configs as legacy published profiles.
6. Build a canonical read adapter for one entity chain. Start with Event -> Sample -> Tissue for a few projects. Preserve current query/export behavior.
7. Add BCID regression tests. Use representative existing records to prove old ARKs still resolve to the same project, expedition, entity, and record.
8. Add validation comparison tests. Run current validation and standards/profile validation side by side, with differences reported but not enforced.
9. Benchmark current query/report paths. Capture baseline latency and plans for common project, public, entity, full-text, and reporting queries before considering storage changes.
10. Run a representative migration dry-run against `dev-seed.tar.gz`. Verify row counts, parent links, BCIDs, exports, and representative API responses.

Explicit Answers to Planning Questions

1. What is the actual current GEOME physical data model?

Public relational tables store users, networks, projects, configurations, expeditions, BCID roots, memberships, templates, SRA submissions, and auth state. Each network also has a PostgreSQL schema named `network_<id>`. Network entities are separate physical tables under that schema. Record scientific metadata is stored as JSONB in each entity table.

2. Which parts are dynamically driven by network/project configuration?

Network config drives entity tables, entity relationships, fields, worksheets, record types, default rules, lists, and queryable entity/field metadata. Project config overlays the network config to choose fields, project rules, lists, and presentation/validation details for a project or reusable team-like config.

3. Which concepts are necessities versus historical artifacts?

Necessary concepts: projects, users/access, expeditions/upload groupings, persistent identifiers, record validation, exports, public/private visibility, and parent-child material lineage. Historical or overgrown concepts: teams as `network_approved` configs, network config as physical schema generator, hard-coded `network_1` scripts, and specialized entity classes where a typed resource/material model would be cleaner.

4. Could GEOME reasonably operate with a single canonical backend entity model?

Yes, but not as a single-step rewrite. The current JSONB storage makes this plausible, but BCIDs, exports, validation, and query behavior require an adapter and regression tests before storage changes.

5. Should teams disappear as backend schema-defining objects?

Yes. Teams/communities should not define physical backend schema. They should define profiles: required fields, optional fields, validation, Mixture package, UI organization, and community terms.

6. Can teams/configurations become validation profiles?

Yes. Existing `project_configurations` are already profile-like overlays. The model needs clearer naming, versioning, and separation from network table creation.

7. What should Mixture control?

Mixture should control applicable scientific metadata terminology, definitions, checklists, environmental extensions, combinations, selected controlled values, syntax guidance, and versioned validation inputs.

8. What should remain GEOME-specific?

BCIDs, projects, access control, expeditions, material relationships, publication workflow, Local Contexts, permits, media/files, external lab links, derived-data resources, profile assignment, and UI/template behavior.

9. Is relational + JSONB appropriate for this workload?

Yes, with discipline. GEOME already uses JSONB effectively for flexible scientific fields. The relational core must continue to hold identity, relationships, access, publication, project/expedition scope, and resource

linkage. Frequently queried JSONB fields should get targeted indexes or promoted columns.

10. How should entity relationships be modeled?

Keep legacy parent identifiers during migration. Long term, use relational entity ids and a typed relationship table for derivation, collection, measurement, media attachment, and external-resource links. Preserve old `conceptAlias/parent_identifier` views for compatibility.

11. How should scientific terms be versioned?

Store term source, source version, term id, term name, and profile version. Projects should be tied to explicit profile versions. New MxS releases should create migration reports, not silently alter historical validation.

12. How should old project configurations continue working?

Keep a legacy adapter that reconstructs effective project config from `PersistedProjectConfig` plus the relevant network config. Snapshot effective profile/config versions before changing validation or storage semantics.

13. What is the safest migration sequence?

Inventory first, then standards registry, then canonical read abstraction, then profiles, then storage migration if justified, then legacy cleanup.

14. What parts of the proposal should change based on the existing codebase?

Focus less on introducing JSONB, because GEOME already uses it. Focus more on decoupling physical table creation from network/community configuration. Distinguish expeditions from sampling Events. Treat MxS as a versioned standards source, not the entire backend model. Preserve current project-config overlay behavior while turning it into explicit profile versioning.